ITNEXT

ITNEXT is a platform for IT developers & software engineers to share knowledge, connect, collaborate, learn and experience next-gen technologies.

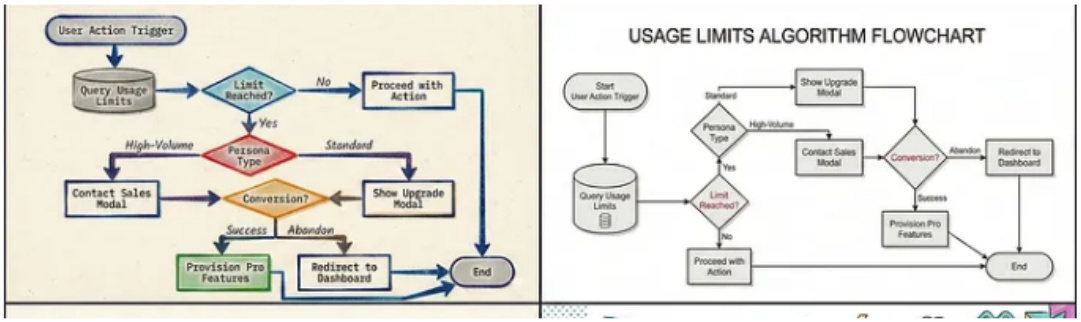
Follow publication

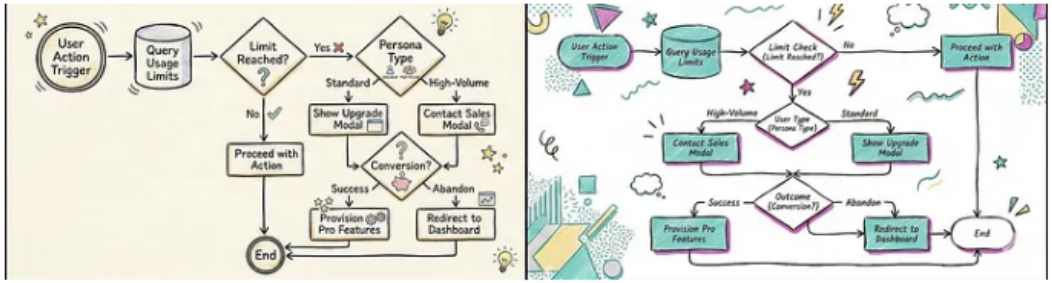
Stop Using Ugly Charts: How to Build Pro-Level Diagrams with Mermaid.js and Nano Banana 2 🚀

Dmitry Kostyuk Follow 6 min read · 1 day ago

59

Google released **Nano Banana 2** a few days ago, and it's a total game-changer for how we visualize logic. People are already dreaming up wild use cases, but as a developer who lives in architecture diagrams, I've found its true "killer app." Creating diagrams. With a twist. 💡





Nothing beats a well-presented flow chart for clarity, but getting them to look "investor-ready" usually involves hours of manual dragging and dropping.

Nightmare.

My new workflow is two-fold: I use **Gemini** to generate and validate a rock-solid **Mermaid.js** diagram, then I feed that structural "source of truth" into **Nano Banana 2** to render a professional, high-fidelity masterpiece. It's the perfect marriage of deterministic logic and AI-driven aesthetics.

Powerful.

Mermaid.js is a JavaScript-based charting and transitioning tool that renders Markdown-inspired text definitions into dynamic diagrams.

Instead of fighting with a mouse, you write code. It is efficient. It is version-controllable. Because it uses a text-to-diagram approach, you can keep your documentation right next to your code in your repository.

Look at how simple a flowchart is to define:

```
graph TD
    A[Start] --> B{Is it working?}
```

```
    B -- Yes --> C[Celebrate]
    B -- No --> D[Debug]
    D --> B
```

You can render this instantly in several ways. The Mermaid Live Editor is the go-to for a quick web-based preview. If you are serious about your workflow, most decent Markdown editors, in my case **Neovim** and **Obsidian,** handle it natively. Even **GitHub** renders Mermaid blocks automatically in your README files.

Why bother with this intermediate step before jumping into Nano Banana 2?

Validation.

Mermaid.js is **deterministic**. It forces you to define the logic strictly. By validating the structure here first, you ensure the underlying "truth" of your diagram is correct before you let the AI get creative with the aesthetics. It also acts as a perfect structural blueprint for the image generator, reducing the "hallucination" risk where the AI might forget a crucial decision branch. Plus, it's much faster to fix a typo in a text string than to re-generate a complex image five times.

Let's build a real-world example.

Feature: SaaS "Freemium to Pro" Upgrade Workflow

We want to design a workflow that nudges users toward a paid plan. The logic needs to be sharp. We aren't just asking for money; we are directing the user based on their specific behavior.

The Logic Breakdown

To create a compelling visual, we split the logic into four distinct phases:

- **The Trigger:** A user initiates a restricted action, such as “Invite Teammate.”
- **The Validation Gate:** A database query checks if the Freemium limit is hit.
- **The Persona Branch:** We segment the user. High-volume users get a “Contact Sales” nudge for Enterprise potential, while standard users see an automated “Upgrade Modal.”
- **The Resolution:** The path ends in provisioning ora redirect to the dashboard.

We use a specific prompt to get Gemini to handle the heavy lifting of the Mermaid syntax.

```
# Role

You are a Senior Systems Architect and Data Visualization Specialist proficient

# Context

The user has a functional description of a software feature, business process, o

# Task

Based on the user description provided below, generate a Mermaid.js code block a

# Constraints

- **Direction:** Default to `graph TD` (Top-Down) unless `graph LR` (Left-to-Rig
- **Node Shapes:** Use appropriate Markdown shapes:
    - `((Start/End))` for terminals.
    - `{Decision}` for conditionals.
    - `[Process]` for standard actions.
    - `[(Database)]` for data storage/retrieval.
```

```
- **Styling:** Use `style` tags at the bottom of the code to highlight "Success"
- **Readability:** Keep node labels concise (3-5 words max).

# Methodology

1. **Identify the Entities:** List the actors and systems involved.
2. **Define the Happy Path:** Map the primary route to success first.
3. **Map the Edge Cases:** Identify where the logic branches for errors, denials
4. **Synthesize:** Write the Mermaid code using clear, alphanumeric IDs (e.g., `

# Output Format

1. **The Logic Summary:** A 2-sentence breakdown of what the chart represents.
2. **Mermaid Code Block:** The copy-pasteable code.
3. **Customization Tip:** One tip on how the user can tweak the visual flow.
4. **Styling:** DO NOT include any styles and stick to the default

# User Description

[INSERT YOUR LOGIC DESCRIPTION HERE]
```

The resulting Mermaid code looks like this:

```
graph TD
    %% Node Definitions
    Start((User Action Trigger)) --> CheckDB[(Query Usage Limits)]
    CheckDB --> LimitCheck{Limit Reached?}

    %% The Happy Path / Validation
    LimitCheck -- No --> Allow[Proceed with Action]
    LimitCheck -- Yes --> UserType{Persona Type}

    %% Branching Logic
    UserType -- High-Volume --> Sales[Contact Sales Modal]
    UserType -- Standard --> Modal[Show Upgrade Modal]

    %% Decision Points
    Sales --> Outcome{Conversion?}
    Modal -->Outcome

    %% Terminal States
    Outcome -- Success --> Provision[Provision Pro Features]
```

```
    Outcome -- Abandon --> Dashboard[Redirect to Dashboard]

    Provision --> End((End))
    Dashboard --> End
    Allow --> End
```

Drop this into the Mermaid Live Editor to confirm the flow is solid.

Keep iterating on the text until the logic is flawless. Don't move on until the flow is right. Non-negotiable.

Generating the Final Diagram with Nano Banana 2

Now for the fun part. We have the skeleton; now we need the skin.

Get Dmitry Kostyuk's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

When moving to Nano Banana 2, style is everything. You want a look that matches your presentation's vibe. Think about **Swiss Modernism** for something clean and authoritative, or **Neo-Memphis** if you want something bold and vibrant. If you're going for a technical, "under-the-hood" feel, **Blueprint** or **Isometric 3D** works wonders.

Use this prompt template to bridge the gap:

```
Generate an algorithm flow chart based on the logic of the Mermaid.js logic prov

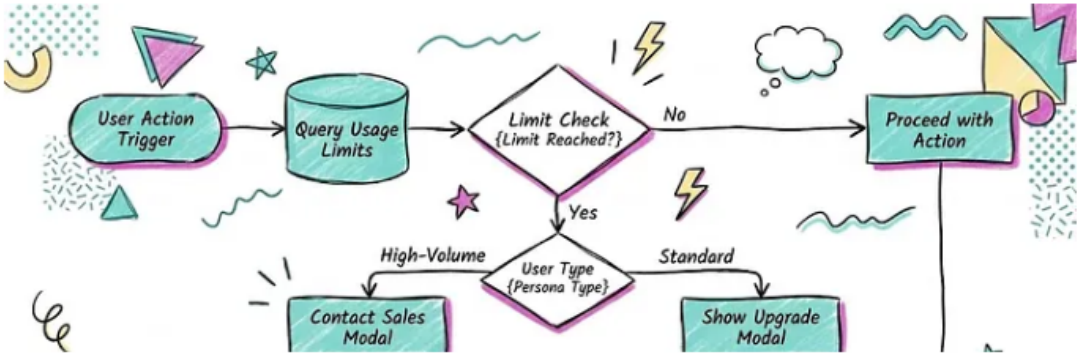
```mermaid
[MERMAID CODE GOES HERE]
```

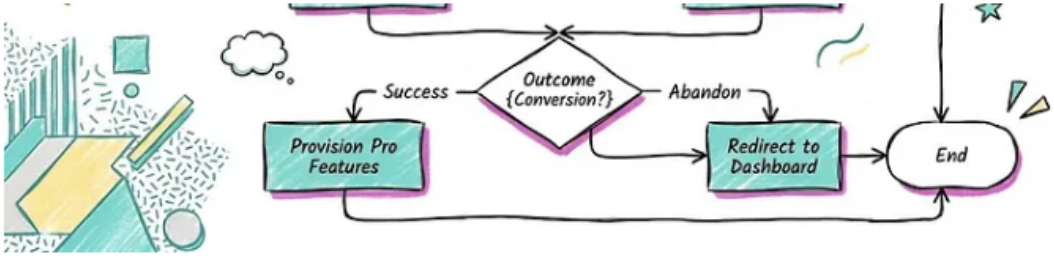
Use standard flowchart conventions: rectangles for processes, diamonds for decis

Style: Neo-Memphis, white background.

Aspect ratio: 16:9.
```

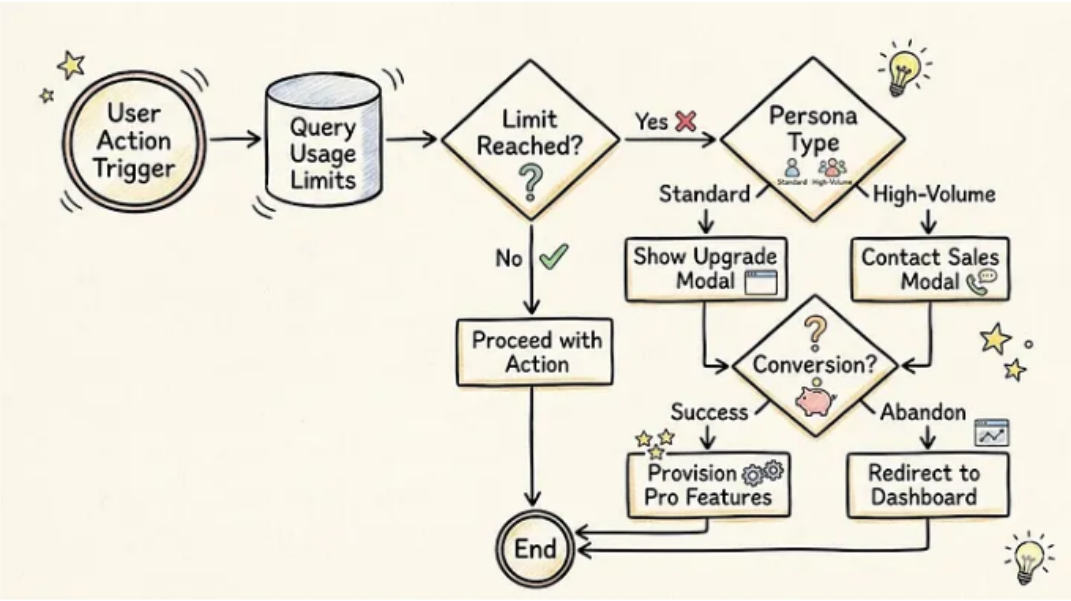
The result is night and day compared to a standard black-and-white chart.



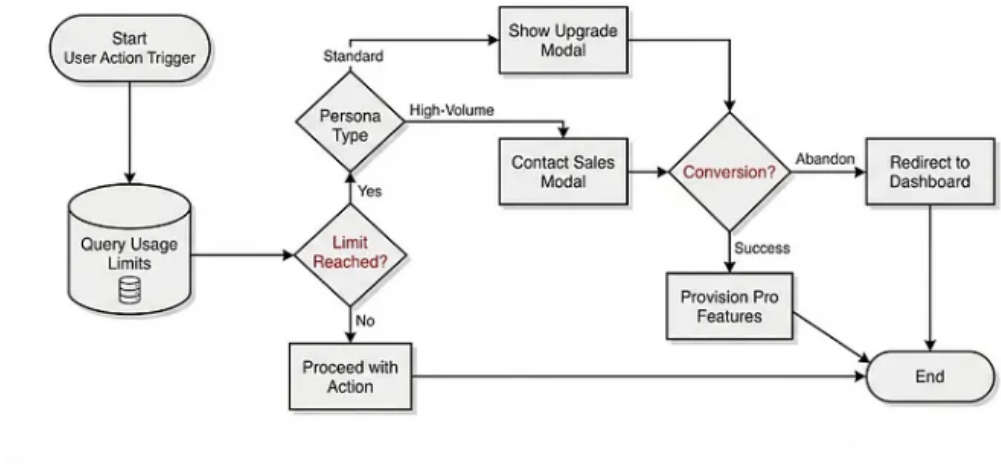


Want to change the mood? Just swap the style tag.

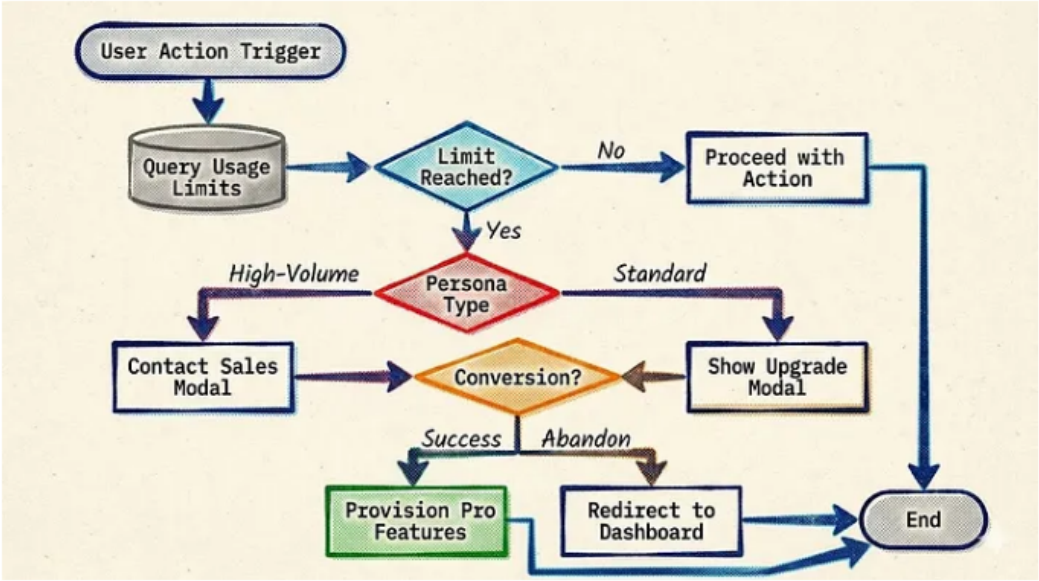
Comic Style/Hand-Drawn "Sketchnote":



Swiss Modernism:



Grainy Risograph:



Some other styles you might want to try out are:

- Isometric 3D
- Corporate Flat 2.0

- Blueprint/Schematic
- Paper Cut-Out/Collage
- Tufte-esque

Elevate Your Visuals

Boring charts are a choice.

With Nano Banana 2 and Mermaid.js, you have a professional design studio at your fingertips. You get the precision of code-based logic combined with the aesthetic flair of a top-tier illustrator. Stop settling for default shapes and start building diagrams that actually capture attention.

About Me

Get my book **Kickstart Google Apps Script**
I am an AI-first Google Cloud and Workspace developer and a Workspace Google Developer Expert (GDE), CTO and co-founder of Mentormatic, and founder of Wurkpaces.dev

Mermaid JavaScript Markdown Nanonbanana2 Gemini